MRC Lecture 2 — Derivation Guide 1

The Neuron and the Forward Pass

Sai T. Reddy — ETH Zurich / BIIE

1. Motivation

The transformer attention mechanism — the subject of this lecture — is built from neural network layers. Before deriving attention, we need to understand the building block from which all neural networks are constructed: the **artificial neuron**. A single neuron computes a dot product followed by a nonlinear activation. A neural network is a composition of many neurons organized into layers. The **forward pass** is the process of computing the network's output from its input — one layer at a time, from input to output.

This guide derives the neuron computation, works through a forward pass in a two-layer network with specific numbers, and connects the computation to the dot product from Lecture 1.

2. The Artificial Neuron

Definition. A single artificial neuron with n inputs computes:

$$y = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right) = \sigma(w \cdot x + b)$$

where:

- $x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ is the input vector
- $w = (w_1, w_2, \ldots, w_n) \in \mathbb{R}^n$ is the weight vector (learned)
- $b \in \mathbb{R}$ is the bias (learned)
- $\sigma : \mathbb{R} \to \mathbb{R}$ is a nonlinear activation function
- $y \in \mathbb{R}$ is the scalar output

Connection to Lecture 1. The core operation inside the neuron is the dot product $w \cdot x$. The weight vector w encodes "what the neuron is looking for" — it defines a direction in input space. The dot product measures how well the input x aligns with that direction (Derivation Guide 1, §6). The bias b shifts the activation threshold. The nonlinearity σ converts the linear score into a bounded or rectified output.

Interpretation. The neuron asks: "How similar is this input to my learned template?" If the dot product $w \cdot x$ is large and positive (input aligns with the template), the neuron activates strongly. If the dot product is negative (input opposes the template), the neuron is suppressed. The bias b controls the baseline — how much evidence is needed before the neuron activates.

3. Worked Example 1: A Single Neuron

Problem. A neuron has weights $w = (0.5, -0.3, 0.8)$, bias $b = 0.1$, and uses the ReLU activation function $\sigma(z) = max(0, z)$. Compute the output for input $x = (1.0, 2.0, 0.5)$.

Step 1. Compute the dot product (the "pre-activation"):

$$w \cdot x = 0.5(1.0) + (-0.3)(2.0) + 0.8(0.5) = 0.5 - 0.6 + 0.4 = 0.3$$

Step 2. Add the bias:

$$z = w \cdot x + b = 0.3 + 0.1 = 0.4$$

Step 3. Apply the activation function:

$$y = \mathrm{ReLU}(0.4) = max(0, 0.4) = 0.4$$

The pre-activation $z = 0.4$ is positive, so ReLU passes it through unchanged. The neuron's output is 0.4.

What if the input were different? For $x' = (-1.0, 3.0, -0.5)$:

$$w \cdot x' = 0.5(-1.0) + (-0.3)(3.0) + 0.8(-0.5) = -0.5 - 0.9 - 0.4 = -1.8$$

$$z' = -1.8 + 0.1 = -1.7$$

$$y' = \text{ReLU}(-1.7) = max(0, -1.7) = 0$$

The pre-activation is negative, so ReLU outputs zero — the neuron is "off." This input does not align with the neuron's learned template.

4. From One Neuron to a Layer

A single neuron produces a scalar output. A **layer** consists of m neurons, each with its own weight vector and bias, all operating on the same input. Together they produce an m-dimensional output vector.

Definition. A layer of m neurons with input $x \in \mathbb{R}^n$ computes:

$$h = \sigma(Wx + b)$$

where:

- $W \in \mathbb{R}^{m \times n}$ is the weight matrix (row j is the weight vector of neuron j)
- $b \in \mathbb{R}^m$ is the bias vector
- σ is applied element-wise to each component of $Wx + b$
- $h \in \mathbb{R}^m$ is the output vector

Connection to Lecture 1. The matrix-vector product Wx computes m dot products in parallel — one per neuron. Row j of W is the weight vector of neuron j, and $(Wx)_j = w_j \cdot x$. This is the same principle as the similarity matrix $S = Q K^T$ from Derivation Guide 2 of Lecture 1, where each row of the output is a dot product between a query and multiple keys. Here, each row is a dot product between a neuron's weights and the input.

5. The Forward Pass: A Two-Layer Network

Architecture. Consider a network with:

- Input: $x \in \mathbb{R}^3$ (3 features)
- Hidden layer: 4 neurons with ReLU activation
- Output layer: 2 neurons with no activation (linear output)

The network computes:

$$h = \text{ReLU}(W_1 x + b_1) \text{ (hidden layer)}$$

$$y = W_2 h + b_2 \text{ (output layer)}$$

Dimensions:

Quantity	Shape	Meaning
x	3×1	Input (3 features)
W_1	4×3	Hidden layer weights (4 neurons, 3 inputs each)
b_1	4×1	Hidden layer biases
h	4×1	Hidden layer output (4 features)
W_2	2×4	Output layer weights (2 neurons, 4 inputs each)
b_2	2×1	Output layer biases
y	2×1	Final output (2 values)

Parameter count: W_1 has $4 \times 3 = 12$ parameters, b_1 has 4, W_2 has $2 \times 4 = 8$, b_2 has 2. Total: $12 + 4 + 8 + 2 = 26$ learned parameters.

6. Worked Example 2: Full Forward Pass

Problem. Compute the output of the two-layer network with the following parameters:

$$W_1 = \begin{pmatrix} 0.2 & 0.4 & -0.5 \\ -0.3 & 0.1 & 0.6 \\ 0.7 & -0.2 & 0.1 \\ 0.0 & 0.5 & 0.3 \end{pmatrix}, b_1 = \begin{pmatrix} 0.1 \\ -0.1 \\ 0.0 \\ 0.2 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0.3 & -0.4 & 0.5 & 0.1 \\ -0.2 & 0.6 & 0.1 & -0.3 \end{pmatrix}, b_2 = \begin{pmatrix} 0.0 \\ 0.1 \end{pmatrix}$$

Input: $x = (1.0, -0.5, 2.0)^T$.

Step 1: Compute the hidden layer pre-activations $z_1 = W_1 x + b_1$.

Each component is a dot product of a row of W_1 with x, plus the corresponding bias:

$$(z_1)_1 = 0.2(1.0) + 0.4(-0.5) + (-0.5)(2.0) + 0.1 = 0.2 - 0.2 - 1.0 + 0.1 = -0.9$$
$$(z_1)_2 = (-0.3)(1.0) + 0.1(-0.5) + 0.6(2.0) + (-0.1) = -0.3 - 0.05 + 1.2 - 0.1 = 0.75$$
$$(z_1)_3 = 0.7(1.0) + (-0.2)(-0.5) + 0.1(2.0) + 0.0 = 0.7 + 0.1 + 0.2 + 0.0 = 1.0$$
$$(z_1)_4 = 0.0(1.0) + 0.5(-0.5) + 0.3(2.0) + 0.2 = 0.0 - 0.25 + 0.6 + 0.2 = 0.55$$
$$z_1 = (-0.9, 0.75, 1.0, 0.55)^T$$

Step 2: Apply ReLU activation.

$$h = \text{ReLU}(z_1) = (max(0, -0.9), max(0, 0.75), max(0, 1.0), max(0, 0.55))^T$$
$$h = (0, 0.75, 1.0, 0.55)^T$$

Neuron 1 is "off" (pre-activation was negative). Neurons 2, 3, 4 are active. This is the **sparsity** property of ReLU — some neurons are inactive for a given input, which makes the representation sparse and efficient.

Step 3: Compute the output layer $y = W_2 h + b_2$.

$$y_1 = 0.3(0) + (-0.4)(0.75) + 0.5(1.0) + 0.1(0.55) + 0.0$$
$$= 0 - 0.3 + 0.5 + 0.055 + 0 = 0.255$$
$$y_2 = (-0.2)(0) + 0.6(0.75) + 0.1(1.0) + (-0.3)(0.55) + 0.1$$
$$= 0 + 0.45 + 0.1 - 0.165 + 0.1 = 0.485$$
$$y = (0.255, 0.485)^T$$

Summary of the forward pass:

$$x = \begin{pmatrix} 1.0 \\ -0.5 \\ 2.0 \end{pmatrix} \xrightarrow{W_1, b_1} z_1 = \begin{pmatrix} -0.9 \\ 0.75 \\ 1.0 \\ 0.55 \end{pmatrix} \xrightarrow{\text{ReLU}} h = \begin{pmatrix} 0 \\ 0.75 \\ 1.0 \\ 0.55 \end{pmatrix} \xrightarrow{W_2, b_2} y = \begin{pmatrix} 0.255 \\ 0.485 \end{pmatrix}$$

The forward pass is a sequence of matrix multiplications (dot products in parallel), bias additions, and nonlinear activations. Nothing else. The entire computation is built from the operations of Lecture 1.

7. Why Nonlinearity Is Essential

Claim. Without nonlinear activations, any multi-layer network collapses to a single linear transformation.

Proof. Consider a two-layer network without activations:

$$y = W_2(W_1 x + b_1) + b_2 = W_2 W_1 x + W_2 b_1 + b_2$$

Define $W' = W_2 W_1$ and $b' = W_2 b_1 + b_2$:

$$y = W' x + b'$$

This is a single linear transformation — equivalent to a one-layer network. The second layer added no representational power. ∎

Implication. Without nonlinearity, depth is useless. No matter how many layers you stack, the result is always a linear function of the input. Linear functions can only separate data with straight lines (hyperplanes). Most real-world problems — including molecular recognition — are nonlinear. The activation function is what allows each layer to carve out nonlinear decision boundaries, and it is what makes depth useful.

For the same example: Without ReLU, the network would compute $y = W_2 W_1 x + W_2 b_1 + b_2$. This is a single 2×3 matrix times x plus a bias — a linear function that can only learn linear relationships. With ReLU, neuron 1 was shut off (output = 0), creating a piecewise linear function that depends on which neurons are active. Different inputs activate different subsets of neurons, producing different effective linear functions in different regions of input space. This piecewise linearity is what gives ReLU networks their approximation power.

8. Training: How the Parameters Are Learned

The weights W_1, b_1, W_2, b_2 are not hand-designed — they are learned from data by minimizing a loss function using gradient descent.

The training loop:

1. **Forward pass:** Compute the output y for a training input x using the current parameters.
2. **Loss computation:** Compare y to the known correct output y^* using a loss function $L(y, y^*)$. Common choices: mean squared error for regression, cross-entropy for classification.
3. **Backward pass (backpropagation):** Compute the gradient of the loss with respect to every parameter: $\partial L/\partial W_1$, $\partial L/\partial b_1$, $\partial L/\partial W_2$, $\partial L/\partial b_2$. This is done by the chain rule, propagating gradients backward from the loss through each layer.
4. **Parameter update:** Adjust each parameter in the direction that reduces the loss:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

where $\eta > 0$ is the learning rate.

5. **Repeat** for many training examples (epochs) until the loss converges.

Key insight for MRC. The loss function determines what the network learns. In standard supervised learning, the loss is prediction error. In contrastive learning (Lecture 3), the loss is InfoNCE — the negative log-likelihood of the convergence equation. The choice of loss function is not arbitrary; for SFMs, it is prescribed by the physics.

9. Depth and Hierarchical Feature Learning

Why deeper networks are more powerful. Each layer of a deep network transforms the representation. Early layers learn simple, local features. Later layers compose these into increasingly complex, abstract features.

Example from computer vision (LeCun et al., 1989):

- Layer 1: detects edges (horizontal, vertical, diagonal)
- Layer 2: combines edges into textures and corners
- Layer 3: combines textures into object parts (eyes, wheels)
- Layer 4: combines parts into full objects (faces, cars)

Example from protein language models (Rives et al., 2021):

- Early layers: amino acid identity and local sequence context
- Middle layers: secondary structure elements (helices, sheets)
- Late layers: tertiary contacts and functional motifs

- Attention heads: capture residue-residue contact maps without ever seeing a 3D structure

The universal approximation theorem (Hornik et al., 1989) guarantees that even a single hidden layer can approximate any continuous function, given sufficient width. But the required width may be exponentially large. Deep networks achieve the same approximation power with polynomially many parameters by composing features hierarchically. This efficiency is why modern architectures (transformers, ResNets) use dozens to hundreds of layers.

10. Summary

Concept	Definition	Connection to Lecture 1
Single neuron	$y = \sigma(w \cdot x + b)$	Core operation is the dot product
Layer	$h = \sigma(Wx + b)$	Matrix-vector product = parallel dot products
Forward pass	Input → (Wx + b) → σ → (Wx + b) → σ → Output	Sequential matrix multiplications + activations
ReLU	$max(0, z)$	Creates sparse, piecewise-linear representations
Nonlinearity	Essential for depth to add power	Without it, multi-layer = single linear layer
Backpropagation	Chain rule computes $\partial L / \partial W$	Gradient descent adjusts parameters to minimize loss
Depth	Hierarchical feature composition	Early layers = simple features; late layers = abstract features

11. Checkpoint Questions

Q1. A neuron has weights $w = (1, -1, 2)$, bias $b = -0.5$, and uses ReLU. Compute the output for $x = (0.5, 1.0, 0.3)$.

Answer: $w \cdot x = 0.5 - 1.0 + 0.6 = 0.1$. $z = 0.1 + (-0.5) = -0.4$. $y = \text{ReLU}(-0.4) = 0$. The neuron is off — the input does not sufficiently match the neuron's template.

Q2. A layer has weight matrix $W \in \mathbb{R}^{8 \times 512}$ and bias $b \in \mathbb{R}^8$. What is the input dimension? What is the output dimension? How many learnable parameters does this layer have?

Answer: Input dimension: 512. Output dimension: 8. Parameters: $8 \times 512 + 8 = 4,104$. Each of the 8 neurons computes a dot product with the 512-dimensional input.

Q3. Explain why the forward pass of a neural network is built entirely from dot products and nonlinear activations.

Answer: Each layer computes $h = \sigma(Wx + b)$. The matrix-vector product Wx computes one dot product per neuron (row of W dotted with x). The bias shifts the result. The activation applies a nonlinearity element-wise. The forward pass through L layers is just L repetitions of this: dot products (via matrix multiplication), bias addition, nonlinearity. Every other operation in a neural network — attention, convolution, normalization — is built from these primitives.

Q4. In the worked example (§6), neuron 1 of the hidden layer was "off" (output = 0) because its pre-activation was negative. What does this mean for the subsequent computation?

Answer: Because $h_1 = 0$, the first column of W_2 is multiplied by zero and contributes nothing to the output. The output depends only on neurons 2, 3, and 4. Effectively, the network is using a different subset of its parameters for this particular input. Different inputs activate different subsets of neurons, which is how the network implements different functions in different regions of input space. This input-dependent parameter selection is the source of ReLU networks' expressive power.

12. Connection to Future Lectures

- **Lecture 2, next derivation (WB2):** The transformer attention mechanism is a specific computation: $\text{softmax}\left(Q K^T / \sqrt{d_k}\right) V$. The Q, K, V matrices are computed by linear

projections (layers without activation): $Q = X\,W_Q$. The attention output passes through a feed-forward network (two layers with ReLU), which is the standard neural network layer derived in this guide.

- **Lecture 3:** Protein language models (ESM-2) are deep transformer encoders — dozens of layers of self-attention and feed-forward networks. The forward pass through ESM-2 produces contextual residue embeddings. The final embedding is used as input to the SFM.
- **Lecture 7:** The SFM architecture consists of two encoders (deep transformers) whose outputs are compared via a dot product. The encoders are trained end-to-end by backpropagation with the InfoNCE loss — the same training loop described in §8, with the loss function prescribed by the convergence equation.